

I Benchmarked Every Open-Source Voice AI Engine at 100 Concurrent Calls. Here's What Actually Scales.

7 min read · May 14, 2026



Nakshatra Kanchan



Listen



Share



More

ASR, LLM, TTS — tested under real load on an A100 80GB. The results are not what I expected.

Most tutorials show you how to chain speech-to-text to an LLM to a TTS engine, run a single request, and ship it. Then someone puts 100 simultaneous callers on it and the whole thing collapses. I spent a week building a full-stack inference load harness and running it against every serious open-source ASR, LLM, and TTS engine on a NVIDIA A100-SXM4-80GB — across concurrency levels of 1, 5, 10, 25, 50, and 100, 3 rounds each, measuring latency percentiles at every stage.

Here's what I found.

The Stack Under Test

A real-time voice AI system has three stages in series:



incoming **audio** → [ASR] → transcript → [LLM] → response text → [TTS] → **audio** out

The latency budget for something that feels like a real phone call:

- ASR (full utterance finalize): < 600 ms
- LLM first token: < 800 ms
- TTS first audio byte (TTFB): < 250 ms
- End-to-end: < 2.0 s at c=1, < 3.5 s at c=100

ASR candidates: `nvidia/parakeet-tdt-0.6b-v3` (NeMo FastAPI), `nvidia/canary-1b-flash` (NeMo FastAPI), `Voxtral-Mini-3B-2507` (vLLM `/v1/audio/transcriptions`), Streaming Zipformer (`sherpa-onnx`), `faster-whisper large-v3-turbo` (baseline)

LLM candidates: `Qwen3-8B bfloat16` (vLLM), `Qwen2.5-7B-Instruct` (vLLM, baseline)

TTS candidates: `Kokoro-82M` (`ghcr.io/remsky/kokoro-fastapi-gpu`), `StyleTTS2`, `Chatterbox` (Resemble AI), `Qwen3-TTS-1.7B`, `Piper` (baseline)

The Benchmark Harness

The load test uses `httpx` with `asyncio.gather` — no threading, just async concurrency fan-out. For each stage:

- **ASR:** POST a 1s synthetic WAV to `/asr`, measure wall time
- **LLM:** Stream a chat completion, measure TTFT and full wall time
- **TTS:** POST text to `/v1/audio/speech`, drain response bytes, measure TTFB and wall time

At c=100 this produces 300 requests per stage and 300 full E2E chains.

One important implementation note on TTS streaming: `httpx's aiter_bytes()` is a single-use iterator. Calling it twice — once to detect the first byte and again to drain

— raises `StreamConsumed` on the second call, turning every TTS measurement into a false failure with a 100% error rate. The fix is a single drain loop:

```
# Wrong: two separate aiter_bytes() calls
chunks = resp.aiter_bytes()
while ...: chunk = await chunks.__anext__()
chunks2 = resp.aiter_bytes() # StreamConsumed ← silent failure
```

```
# Correct: one drain loop, TTFB recorded on first non-empty chunk
async for chunk in resp.aiter_bytes():
    if ttfb is None and chunk:
        ttfb = time.perf_counter() - t0
    if not drain_response:
        break
```

This bug is subtle enough that I’d estimate a lot of benchmark code in the wild has the same issue.

LLM: vLLM Scales Beautifully

Qwen3-8B on vLLM with PagedAttention + continuous batching scales to 100 concurrent calls with near-linear TTFT growth:

Concurrency	TTFT p50	Wall p50	Success
1	23 ms	730 ms	100%
5	37 ms	763 ms	100%
10	58 ms	785 ms	100%
25	94 ms	862 ms	100%
50	154 ms	968 ms	100%
100	278 ms	1.27 s	100%

278 ms TTFT at 100 concurrent calls, 100% success rate. Full wall time barely moves because continuous batching amortizes decode cost across all active sequences — the KV cache is never the bottleneck since vLLM pages it on demand rather than allocating per-sequence.

The practical implication: **the LLM is not your bottleneck at scale**. Don't buy bigger GPUs or switch models to fix voice latency. The problem is elsewhere.

ASR: Parakeet Wins on Latency, Voxtral Wins on Resilience

Engine	c=1 p50	c=10 p50	c=50 p50	c=100 p50	c=100 success
Parakeet-TDT-0.6B-v3	97 ms	920 ms	4.67 s	5.86 s	99.3%
Canary-1B-Flash	132 ms	1.36 s	6.54 s	10.41 s	91.3%
Voxtral-Mini-3B	190 ms	367 ms	777 ms	1.66 s	100%
Streaming Zipformer	266 ms	2.78 s	14.15 s	21.64 s	78.3%
Whisper large-v3-turbo	451 ms	4.54 s	22.83 s	45.93 s	100%

Parakeet is the fastest single-worker model at low concurrency — 97 ms at c=1 for a 600M parameter model is genuinely impressive. But Voxtral is **3.5× faster at c=100** (1.66 s vs 5.86 s) while hitting 100% success rate.

The reason: Voxtral runs inside vLLM. It inherits continuous batching and PagedAttention, the same infrastructure used by the LLM stage. It's not a single Python process — it's a proper inference server, and the fan-out at high concurrency is handled natively.

Canary-1B-Flash and Zipformer both degrade badly above c=50. Zipformer is streaming-first by design (it emits partial transcripts during audio playback), which an HTTP-batch harness can't fairly capture — a WebSocket-based test would look very different for it.

One sherpa-onnx gotcha: in version 1.10+, `rec.get_result(stream)` returns a plain `str` instead of an object with a `.text` attribute. Every older tutorial will tell you to

call `.text`. This fails silently or crashes with no useful error. The fix:

```
_result = recognizer.get_result(stream)
text = _result if isinstance(_result, str) else getattr(_result, "text", str(_r
```

TTS: The Other Bottleneck

Engine	Params	c=1 TTFB	c=10 TTFB	c=100 TTFB	Notes
Kokoro-82M	82M	129 ms	1.04 s	6.74 s	GPU-accelerated, OpenAI-compatible API
StyleTTS2	~100M	601 ms	3.43 s	20.68 s	Usable; slower than Kokoro
Piper (baseline)	—	~200 ms*	—	9.15 s	CPU-based
Chatterbox	~440M	4.01 s	60.76 s	—	Requires watermarker shim
Qwen3-TTS- 1.7B	1.7B	33.55 s	472.9 s	—	Non-viable for real-time

Piper number from the full baseline stack run, not standalone.



Kokoro-82M wins clearly. At `c=1` it hits 129 ms TTFB — well inside the 250 ms budget — ships as a Docker container, and exposes an OpenAI-compatible `/v1/audio/speech` interface.

StyleTTS2 is the most interesting alternative at 601 ms TTFB, though loading it required two shims: `torch.load(..., weights_only=False)` for PyTorch version compatibility and `nltk.download('punkt_tab')` because the punkt model changed names between NLTK versions.

Chatterbox wouldn't load because `resemble_perth.PerthImplicitWatermarker` is `None` in the current package version. Replacing it with a no-op class before importing fixes the load — but 4 s TTFB at `c=1` and 60 s at `c=10` make it a non-starter for real-time.

Qwen3-TTS at 33.55 s for a single request is a model problem, not a shim problem. It's a 1.7B autoregressive model generating audio tokens sequentially with no streaming decoder. Fine for async tasks, not live calls.

Orpheus-3B never produced output: the model loads, `/healthz` returns 200, but `/v1/audio/speech` returns HTTP 200 with an empty body. The `orpheus-speech` package runs its own internal vLLM instance with a SNAC decoder post-processing step that, in the current version, simply doesn't emit bytes into the HTTP response. Given Kokoro at 129 ms, the debugging investment isn't worth it.

The Architecture Problem at Scale

Here's the end-to-end picture with the best single-instance setup (Parakeet + Qwen3-8B + Kokoro):

Concurrency	E2E p50	E2E p95	Success
1	984 ms	1.07 s	100%
5	1.99 s	2.15 s	100%
10	2.15 s	2.54 s	100%
25	5.38 s	6.42 s	100%
50	9.37 s	13.8 s	100%
100	18.1 s	28.8 s	100%

984 ms at c=1 is excellent. Under 2.5 s through c=10. But 28.8 s P95 at c=100 is an architectural problem, not a tuning problem.

The math is straightforward: Parakeet takes 97 ms per request as a single Python process. At 100 queued requests, the last one waits 9.7 s before it even starts. Kokoro has the same queue. vLLM avoids this because continuous batching runs all active requests through the same forward pass at every decode step. ASR and TTS models don't have an equivalent — they're sequential, not autoregressive engines that can batch arbitrarily.

The fix for Parakeet/Kokoro is replication behind a load balancer:

```
upstream parakeet_pool {
    least_conn;
    server 127.0.0.1:9001;
    server 127.0.0.1:9002;
    server 127.0.0.1:9003;
    server 127.0.0.1:9004;
}
server {
    listen 9000;
    location /asr { proxy_pass http://parakeet_pool; }
}
```

Four Parakeet workers (each ~2 GB VRAM) can serve ~40 concurrent requests under 1 s instead of 1. The same pattern applies to Kokoro. At 80 GB total, this is easily within budget.

For ASR at high concurrency, Voxtral via vLLM is architecturally cleaner — continuous batching handles the fan-out natively, no nginx config required.

Full Voxtral Stack Results

Concurrency	ASR p50	LLM TTFT p50	TTS TTFB p50	E2E p50	E2E p95	Success
1	190 ms	35 ms	132 ms	1.07 s	1.11 s	100%
5	242 ms	51 ms	671 ms	1.94 s	2.32 s	100%
10	367 ms	63 ms	988 ms	2.71 s	3.12 s	100%
25	506 ms	106 ms	1.97 s	5.10 s	6.22 s	100%
50	777 ms	176 ms	3.94 s	10.42 s	12.08 s	100%
100	1.66 s	293 ms	6.82 s	17.71 s	18.92 s	100%

100% success across all six levels. The ASR-to-LLM handoff is tight — at c=100, Voxtral ASR is 1.66 s and LLM TTFT is 293 ms. Both stages are healthy. The remaining bottleneck is Kokoro TTS at 6.82 s TTFB, which blows the E2E budget on its own.

What I'd Run in Production

For ≤ 10 concurrent calls (single-server):

- ASR: Parakeet-TDT-0.6B-v3—97 ms at c=1, stays under 1 s through c=10
- LLM: vLLM + Qwen3-8B — 58 ms TTFT at c=10, 100% success
- TTS: Kokoro-82M — 129 ms TTFB at c=1, 1.04 s at c=10
- E2E at c=10: ~2.15 s p50. **Passes production threshold.**

For 100 concurrent calls:

- ASR: Voxtral via vLLM (scales natively) or Parakeet $\times 4$ replicas behind nginx
- LLM: vLLM + Qwen3-8B (no change needed)
- TTS: Kokoro $\times 4$ containers behind nginx
- **Expected E2E: the 18 s p50 should drop to 5–6 s once TTS is replicated**

Don't bother with:

- Chatterbox: 4 s TTFB at $c=1$ — great for voice quality experiments, not real-time
- Qwen3-TTS: 33 s at $c=1$ — not a real-time model
- Whisper (any variant): Parakeet replaces it at $7.8\times$ the speed; Voxtral at $27\times$

What's Next

The LLM ceiling is probably $c=300$ – 500 before KV cache pressure bites — there's clear headroom at $c=100$. That's worth a dedicated sweep.

The Zipformer case deserves a WebSocket-based harness to test fairly. The HTTP-batch numbers here reflect queuing, not actual streaming latency. For push-to-talk with real audio streaming, Zipformer is worth a proper evaluation.

The TTS gap at high concurrency is the clearest remaining architectural problem. The ideal solution would run TTS inside vLLM the same way Voxtral runs ASR — autoregressive audio token generation with a streaming decoder that emits bytes before the full sequence is complete. Orpheus and Qwen3-TTS are already in that architecture; what's missing is a packaged implementation with a working streaming output path and competitive performance.

All models are publicly available on HuggingFace. The benchmark harness is reproducible on any A100-class GPU (80 GB VRAM recommended for the full stack; ASR-only runs fit in substantially less).

Llm Inference

Text To Speech

Real Time Communication

Voice Ai

Design Systems